



PROGRAMA DE FORMACIÓN MARVELL

Diseño Digital para Sistemas DSP

Ejercicios Módulo 2: HDL — Sistemas Secuenciales y Combinacionales

Profesor: Del Barco, Martín.

Alumnos: Monja, Ernesto Joaquín
Zúñiga, Guillermo Rubén Darío

Año 2026

Índice

1. Introducción	2
2. Desarrollo	2
2.1. Blocking vs Non-Blocking	2
2.2. Registro de 8 Bits con Clock Enable	3
2.3. Contador BCD 0-9 con Enable	4
2.4. FSM Detector de Secuencia “101”	5
2.5. Pipeline 3-etapas con Valid/Ready	6
2.6. Debugging de Latch Oculto	7
3. Conclusiones	9
4. Referencias	10

1. Introducción

El presente informe detalla la resolución y simulación de los ejercicios correspondientes al módulo 2 del programa de formación en *Diseño Digital para Sistemas DSP*. A lo largo del documento, se abordan implementaciones en *Verilog* de sistemas secuenciales y combinacionales, evaluando conceptos como el uso correcto de asignaciones, diseño de máquinas de estado finitas (*FSM*) y técnicas de *debugging* en código *RTL*.

2. Desarrollo

2.1. Blocking vs Non-Blocking

En este primer ejercicio se analizó el comportamiento de las asignaciones bloqueantes (`=`) y no bloqueantes (`<=`) al intentar implementar un rotador circular con tres registros inicializados en $a = 1$, $b = 2$ y $c = 3$ antes del primer flanco. Se simuló el diseño utilizando *iverilog* y se observaron los valores en los primeros flancos de reloj.

A partir de la simulación, se obtuvieron los siguientes resultados para ambos casos en los instantes de 10 [ns] y 20 [ns] , tal como muestra la [Fig. 1](#).

```
>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile tb_casos.vcd opened for output.
```

Tiempo	Caso A (blocking)	Caso B (non-blocking)
reset	a=1 b=2 c=3	a=1 b=2 c=3
10ns	a=2 b=3 c=2	a=2 b=3 c=1
20ns	a=3 b=2 c=3	a=3 b=1 c=2
30ns	a=2 b=3 c=2	a=1 b=2 c=3

```
Observar: Caso B rota circularmente (1,2,3)->(2,3,1)->(3,1,2)->(1,2,3)
          Caso A NO rota -- el orden de = importa y rompe la abstraccion.

tb_casos.v:59: $finish called at 36000 (1ps)
VCD generado: tb_casos.vcd (abrir con: gtkwave tb_casos.vcd)
```

Figura 1: Salida por terminal de la simulación realizada.

Como se puede observar claramente en los resultados, el **Caso B** es el que implementa un rotador circular real, completando la secuencia $(1, 2, 3) \rightarrow (2, 3, 1) \rightarrow (3, 1, 2) \rightarrow (1, 2, 3)$. Por el contrario, el **Caso A** no logra rotar los valores de manera correcta debido a que el orden de las asignaciones bloqueantes importa y rompe la abstracción del hardware.

En conclusión, el estilo correcto para inferir lógica secuencial (como un registro de desplazamiento o rotador circular) es utilizar asignaciones no bloqueantes (`<=`) dentro de bloques `always_ff`.

2.2. Registro de 8 Bits con Clock Enable

Se implementó en Verilog un registro con las siguientes características:

- Entrada y salida de 8 bits.
- Reset asíncrono, activo por bajo, que establece la salida en 0.
- Clock enable síncrono, activo por alto.

Este registro es básicamente un Flip-Flop D, es decir, actualiza la salida al valor de la entrada con cada flanco ascendente de reloj, solo si el CE está activo. Se simuló el diseño con un testbench que aplica 10 vectores y compara la salida **q** con la salida teórica. Los resultados fueron correctos, mostrándose a continuación.

Tiempo				
-----	+-----			
100ns	rst=1	ce=1	d=0	q=0
105ns	rst=1	ce=1	d=2	q=2
115ns	rst=1	ce=1	d=4	q=4
125ns	rst=1	ce=1	d=ff	q=ff
135ns	rst=1	ce=0	d=0	q=ff
145ns	rst=1	ce=0	d=aa	q=ff
155ns	rst=1	ce=0	d=1f	q=ff
160ns	rst=0	ce=0	d=80	q=0
165ns	rst=0	ce=1	d=40	q=0
175ns	rst=0	ce=1	d=20	q=0

- RESULTADOS -

Vector 1: PASS
Vector 2: PASS
Vector 3: PASS
Vector 4: PASS
Vector 5: PASS
Vector 6: PASS
Vector 7: PASS
Vector 8: PASS
Vector 9: PASS
Vector 10: PASS

2.3. Contador BCD 0-9 con Enable

Se diseñó un contador decimal *BCD* de 4 bits que cuenta de 0 a 9 y retorna a 0 (*rollover*). El sistema cuenta con un reset síncrono activo por alto y una señal de habilitación (*en*). Además, se incluyó una salida (*tc*) que se activa combinacionalmente durante un único ciclo de reloj cuando el contador alcanza el valor de 9 y el enable se encuentra activo.

Para la verificación, se elaboró un *testbench* con un reloj de 100 [MHz] y un bloque *auto-checker* integrado. Se ejecutó una simulación de 100 ciclos de reloj continuos con la señal de enable activa, tras los cuales el sistema reportó por consola el estado [PASS], contabilizando exitosamente 10 pulsos exactos de la señal *tc* tal como muestra la Fig. 2.

```
>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile bcd_counter.vcd opened for output.

=====
      RESULTADOS DE LA SIMULACIÓN - BCD COUNTER
=====
Ciclos con EN=1 ejecutados: 100
Pulsos 'tc' detectados:      10
-----
ESTADO: [ PASS ] - El rollover funciona perfecto.
=====

tb_bcd_counter.v:80: $finish called at 1025000 (1ps)
VCD generado: tb_bcd_casos.vcd (abrir con: gtkwave tb_casos.vcd)
```

Figura 2: Salida por terminal de la simulación realizada.

Finalmente, se generó un archivo *VCD* para inspeccionar visualmente las señales mediante la herramienta *GTKWave*. En la Fig. 3 se aprecian las formas de onda resultantes de la simulación.

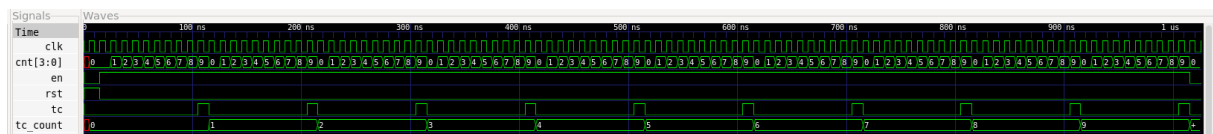


Figura 3: Formas de onda de la simulación del contador *BCD* en *GTKWave*.

A través del diagrama temporal se constata el comportamiento esperado del *HW*. Se observa la correcta inicialización mediante el reset síncrono en el primer flanco de reloj, el ciclo de *rollover* que salta de 9 a 0 sin estados intermedios, y la generación instantánea del pulso *tc* sin desfasajes de reloj. Esto último valida la implementación exitosa de la separación entre la lógica secuencial (el contador) y la lógica combinacional (la bandera de fin de cuenta).

2.4. FSM Detector de Secuencia “101”

Se implementó una *máquina de estados finita* (FSM) que levanta una salida cuando detecta la secuencia “101” en una entrada serial. Por ejemplo, si la entrada es de secuencia 110101 la salida tiene la secuencia 000101.

El detector se comporta según el diagrama de estados de la Fig. 4 y tiene las siguientes características:

- 1 bit de entrada y de salida.
- Reset asíncrono activo por bajo.
- Codificación binaria.
- Sigue el modelo de Moore.

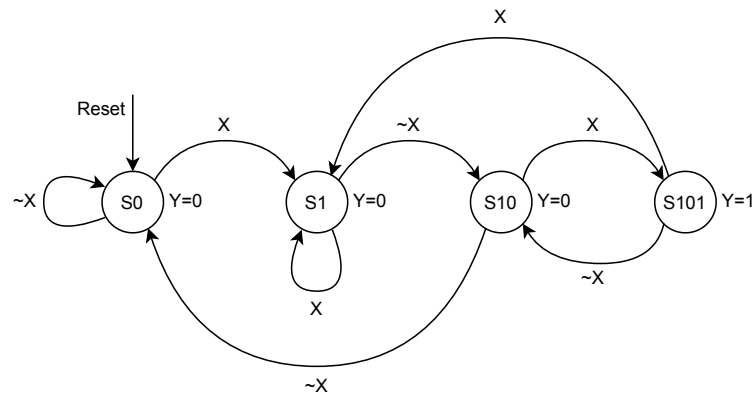


Figura 4: Diagrama de estados de la FSM.

En el testbench produjo la salida a continuación:

Tiempo			
-----	+	-----	-----
115ns	x = 0	y(esperado) = 0	y(obtenido) = 0
125ns	x = 1	y(esperado) = 0	y(obtenido) = 0
135ns	x = 1	y(esperado) = 0	y(obtenido) = 0
145ns	x = 0	y(esperado) = 0	y(obtenido) = 0
155ns	x = 1	y(esperado) = 1	y(obtenido) = 1
165ns	x = 0	y(esperado) = 0	y(obtenido) = 0
175ns	x = 1	y(esperado) = 1	y(obtenido) = 1
185ns	x = 1	y(esperado) = 0	y(obtenido) = 0
195ns	x = 0	y(esperado) = 0	y(obtenido) = 0
205ns	x = 1	y(esperado) = 1	y(obtenido) = 1

215ns	x = 0	y(esperado) = 0	y(obtenido) = 0
225ns	x = 1	y(esperado) = 1	y(obtenido) = 1
235ns	x = 0	y(esperado) = 0	y(obtenido) = 0
245ns	x = 0	y(esperado) = 0	y(obtenido) = 0
255ns	x = 1	y(esperado) = 0	y(obtenido) = 0
265ns	x = 0	y(esperado) = 0	y(obtenido) = 0
275ns	x = 1	y(esperado) = 1	y(obtenido) = 1

Se puede observar que el diseño contempla correctamente todos los casos de prueba y posee la característica de superposición que se solicitó en el ejercicio, es decir, el último 1 de una secuencia “101” es el primer elemento de una próxima secuencia “101” potencial.

2.5. Pipeline 3-etapas con Valid/Ready

En este ejercicio se diseñó un *pipeline* de 3 etapas encargado de calcular la expresión aritmética presentada en la Ec. 1, procesando muestras de 8 bits con signo (**signed**) y utilizando las constantes fijadas por consigna $A = 5$ y $B = 3$.

$$y = ((x + A) * B) \gg 4 \quad (1)$$

La interfaz del módulo se construyó respetando el protocolo de *handshake AXI-Stream* mientras que para implementar la lógica de *back-pressure*, se utilizó la señal de entrada **ready_in** como un *Clock Enable* común para todos los *Flip-Flops* del diseño. De esta forma, cuando el receptor no está listo (**ready_in** = 0), el sistema entra en estado de *stall*, congelando el procesamiento y propagando la condición hacia la entrada mediante **ready_out**.

La validación funcional se realizó mediante un *testbench* equipado con un *golden model* por software y una estructura de datos tipo *FIFO* para gestionar la latencia temporal de las muestras. Se evaluaron 2 escenarios:

- Fase 1 (Régimen ideal): Transmisión continua sin *stalls*. El *throughput* observado se alineó con el rendimiento teórico de 1 muestra/ciclo (procesando 20 muestras en 26 ciclos, contemplando latencias de llenado y vaciado).
- Fase 2 (Régimen con *stalls*): Se introdujeron cortes aleatorios en la señal **ready_in**, verificando que no existiera pérdida ni sobreescritura de datos.

```

>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile pipeline.vcd opened for output.
>>> INICIANDO FASE 1: Transmision sin stalls...
Rendimiento Fase 1 (Ideal): 20 muestras en 26 ciclos.
>>> INICIANDO FASE 2: Transmision con STALLS...
Rendimiento Fase 2 (Con Stalls): 17 muestras en 60 ciclos.

=====
RESULTADOS DE LA SIMULACIÓN - PIPELINE AXI-5
=====
ESTADO: [ PASS ] - El pipeline funciona perfecto.
Throughput ideal teorico: 1.0 muestras/ciclo
=====

tb_pipeline.v:159: $finish called at 875000 (1ps)

```

Figura 5: Salida por terminal de la simulación realizada.

El verificador automático reportó cero errores. En la Fig. 6 se aprecian las formas de onda capturadas durante la simulación.

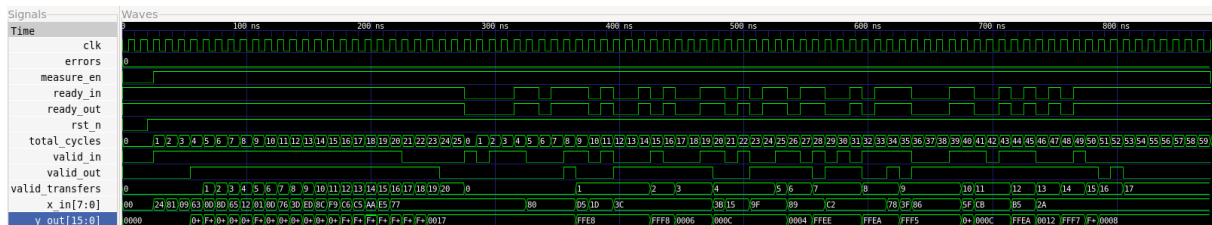


Figura 6: Simulación del pipeline evidenciando la latencia de 3 ciclos y el comportamiento ante *stalls*.

A partir del análisis temporal, se confirma el correcto comportamiento del flujo de datos. Se aprecia claramente el desfase inicial de 3 ciclos hasta la activación de `valid_out`, y cómo las señales y datos internos se mantienen estables cada vez que `ready_in` cae a nivel bajo, comprobando la robustez de la lógica de *back-pressure*.

2.6. Debugging de Latch Oculto

Para este caso, se analizó un código que genera un latch a la hora de la síntesis, es decir, una lógica combinacional que almacena un dato.

```

always @(*) begin
    case (op)
        2'b00: y = a + b;
        2'b01: y = a - b;
        2'b10: y = a & b;
    endcase
end

```


Se trata de una ALU donde uno de los valores del selector de función no realiza ningún cambio, es decir, la salida no está asignada en este valor. Los latches no están permitidos a la hora de diseñar, ya que pueden traer consecuencias como la metaestabilidad.

Se desarrollaron dos archivos en verilog: uno donde se asigna la salida en el caso *default* del case y otro donde se lo asigna justo cuando se entra al bloque *always*. Esta salida es una OR entre a y b. Luego se compararon los tres códigos con un testbench que inyecta 64 casos de entrada para cada valor del selector:

```
for (i = 0; i < 4; i = i + 1) begin
    for (j = 0; j < 64; j = j + 1) begin
        #10
        op = i;
        a = 8'h00 + j;
        b = 8'h01 + j;
    end
end
end
```

Una porción de los resultados se muestra a continuación:

```
a = 3d  b = 3e  op = 2  y(latch) = 3c  y(default) = 3c  y(pre-assign) = 3c
a = 3e  b = 3f  op = 2  y(latch) = 3e  y(default) = 3e  y(pre-assign) = 3e
a = 3f  b = 40  op = 2  y(latch) = 00  y(default) = 00  y(pre-assign) = 00
a = 00  b = 01  op = 3  y(latch) = 00  y(default) = 01  y(pre-assign) = 01
a = 01  b = 02  op = 3  y(latch) = 00  y(default) = 03  y(pre-assign) = 03
a = 02  b = 03  op = 3  y(latch) = 00  y(default) = 03  y(pre-assign) = 03
```

Se observa que el último valor que adquiere y es 00, y cuando el selector op cambia al valor sin asignar, y se mantiene almacenado como 00. Al no haber declarado ninguna logica de reloj, esto implica en la generación de un latch. Este fenómeno no ocurre así con los otros dos casos, donde y sí tiene asignado un valor para ese selector.

Si se sintetizan los tres diseños con Yosys, se puede observar en los reportes la generación del latch. El reporte para el caso de la ALU con latch muestra

```
5.8. Executing PROC_DLATCH pass (convert process syncs to latches).
Latch inferred for signal `alu_bad.\y' from process `alu_bad.$proc$/
home/user/openlane2/designs/alu_bad/src/alu_bad.v:15$1':
$auto$proc_dlatch.cc:433:proc_dlatch$10
```

mientras que en los otros dos casos, el reporte muestra

```
5.8. Executing PROC_DLATCH pass (convert process syncs to latches).
No latch inferred for signal `alu_fix1.\y' from process
`alu_fix1.$proc$/home/ruben/openlane2/designs/alu_fix1/src/
alu_fix1.v:13$1'.
```

3. Conclusiones

Los ejercicios documentados cubren los principales fundamentos del diseño de hardware en RTL. Se observó la importancia de distinguir las asignaciones *bloqueante* y *no bloqueante* para evitar condiciones de carrera. Luego, se desarrollaron distintas arquitecturas digitales tales como un contador BCD, un pipeline con handshake AXI-Stream, etc. y finalmente se realizó un debbuging de un latch oculto. Todo esto fue verificado mediante testbenches, comparando módulos y observando formas de onda resultantes utilizando GTKWave.

4. Referencias

- [1] E. Monja y G. R. D. Zúñiga, *Módulo 2 - Ejercicios*, Repositorio en GitHub, Diseño Digital para Sistemas DSP, Archivos desarrollados para los ejercicios del Módulo N°2, 2026. dirección: https://github.com/ErnestMonja/Diseno_Digital_para_Sistemas_DSP/tree/main/M%C3%B3dulo%20-%20Ejercicios.